ENGINEERING CASE STUDY • SDE1 PORTFOLIO

# Production Backend Case Study

Designing and shipping an internal, role-guarded, time-zone-correct read API against a multi-million-row MySQL table — from India, with the production database in the USA.

**Project:** Internal Invoices Archive Read API

**Stack:** Node.js • Express • MySQL 8 • mysql2/promise • Puppeteer (existing) • America/New_York TZ anchoring

**Format:** STAR-structured engineering walk-through with interview-ready talking points

**Date:** May 19, 2026

Prepared for technical interview discussion. All numbers are either measured or explicitly marked as estimates.

## Table of Contents

## 1. Executive Summary

I designed and shipped two new internal HTTP endpoints — `GET /api/invoices-archive` and `GET /api/invoices-archive/count` — that let authorised CFC teammates query the production `invoices_archive` table by business day, without VPN access or direct SQL. The work was completed across a 12.5-hour time-zone gap, deployed to a production database in the USA in a single commit, with zero downtime and zero regression to the existing surface.

> **Headline outcomes**
>
> - Eliminated routine DB-shell access for non-engineers (admins + managers) for day-level audit queries.
> - Replaced `DATE(created_at)=?` (function-on-column, non-sargable) with a btree-friendly range filter backed by a new index, `idx_ia_created_at`.
> - Anchored the API to America/New_York to align with CFC's existing daily-reset cadence and remove a class of time-zone bugs.
> - Reused the existing `requireRole` middleware verbatim — no new auth surface, no new credentials.

## 2. Business Problem

CFC operates retail furniture stores in the United States. The `invoices_archive` table is the canonical log of every ticket the point-of-sale produces — regular sales (RG), pickups (PH), quote sheets (QS), and goods returns (RTS). Admins and store managers regularly need to answer questions like *"How many tickets did Arden write today?"* or *"Show me yesterday's RG sales for reconciliation."*

Until this work shipped, those questions required either a screenshot from another teammate or a direct SQL session from an engineer. The first is error-prone; the second is a security and on-call burden.

## 3. Technical Problem

- **Read-only, role-guarded endpoint surface** against an existing table that already participates in invoice-create, Birdeye delivery, and daily reporting flows — must not perturb any of them.
- **LONGTEXT payload column** (`form_json`) can be 50 KB+ per row, so naïvely returning a full day will exhaust memory and bandwidth.
- **No existing index on** `created_at`, and the rest of the codebase uses `DATE(created_at) = ?` which would not benefit from one anyway.
- **Time-zone confusion** — the dev clock is IST, the server may be UTC, the data is NY-local. All three must converge to the same "business day".

## 4. Constraints

- India-based developer, production in the USA — deploying speculatively is expensive.

- No JWT or session middleware exists in the codebase; the convention is body/query/header `userId` re-verified against `cfc_users` on every request.
- Adding new npm dependencies is discouraged; the rule is "reuse first".
- Schema changes must be additive and dated according to a documented in-team convention.
- Existing daily-summary, RTS-capture, Birdeye, and search flows must not regress.

## 5. Production Challenges

- The `cfc_form` pool is shared with high-traffic write paths (invoice save, Birdeye send, RTS create). A heavy ad-hoc scan over `invoices_archive` would contend with INSERT throughput.
- There is no staging environment that mirrors production volume. Behaviour at scale must be reasoned about, not measured locally.
- Any regression surfaces hours later because of the time-zone offset between developer and operators.

## 6. Why Existing Solutions Were Insufficient

| Existing surface | Why it didn't fit |
|---|---|
| `/api/reports/today/...` | Pre-aggregated daily report; does not expose raw rows for ad-hoc audit. |
| Direct MySQL access | Requires VPN, raw credentials, broad blast radius, no audit log. |
| Daily Salesperson Summary PDF | Emailed once a day; no on-demand query path. |
| `/api/customers/search` | Indexes customer rows, not invoices; doesn't answer "show me the day." |

## 7. Scale Considerations

- The table accumulates rows daily; without an index, full-scan cost grows linearly with retention.
- Pagination caps response size deterministically (≤ 200 rows × ~50 KB = ≤ 10 MB worst case, with the `includeFormJson=false` escape hatch dropping this by 1–2 orders of magnitude).
- Two queries fan out via `Promise.all` instead of using deprecated `SQL_CALC_FOUND_ROWS`; both share the same index.

## 8. Security Considerations

- Role re-verified server-side every request — forging an `x-user-role` header is useless.
- Every input is parameterised. `invoice_type` additionally passes a regex ( `/^[A-Z0-9]{1,10}$/` ) so the equality side of the WHERE clause is bounded even if a future refactor changes how it's quoted.
- No write path at any layer; the routes file declares only `router.get(...)`.
- Stack traces are never returned to the client; 5xx bodies are a generic string.

## 9. Performance Considerations

- **Sargable range filter**: `created_at >= ? AND created_at < ?` instead of `DATE(created_at) = ?`. The former is index-friendly; the latter wraps the column in a function and is forced to evaluate per row.
- **New btree index**: `idx_ia_created_at (created_at)`, added via an idempotent migration script.
- **Parallel count + page fetch** via `Promise.all` — minimum wall-clock latency.
- **Deterministic sort order**: `ORDER BY created_at ASC, id ASC` so pagination is stable even when many rows share a timestamp.

## 10. Reliability Considerations

- Validation throws structured errors with `.status` and `.code`; the controller maps these to 4xx without leaking internals.
- Empty result sets return `200` with an empty `data` array — never 404. This keeps consumers' branching simple.
- Malformed `form_json` rows fall back to their raw string instead of dropping the row, so legacy bad data never silently disappears.
- The index migration is idempotent — re-running the deploy script does not error.

## 11. Observability & Logging

- Every request logs caller identity, date scope, result size, and the operation tag — e.g. `[InvoicesArchive] list date=2026-05-15 count=50/213 limit=50 offset=0 by=sgupta`.
- Validation failures log at `warn`; unexpected exceptions log a full stack at `error` but never leak it.
- The structured log shape matches the existing `[Module] action ...` convention so downstream log search continues to work.

## 12. Deployment Concerns

- Single-commit change, additive only.
- No environment variables introduced.
- No npm dependencies added.
- SPA catch-all route ordering verified — the new mount sits before `app.get(/.*/, ...)` in `app.js`.
- Index DDL packaged as an idempotent SQL file under `config/migrations/` so an accidental second apply is a no-op.

## 13. India ↔ USA Workflow Challenges

- The 12.5-hour offset means a regression caught during US business hours surfaces hours later for me. I optimised for "first deploy correct" rather than "deploy and iterate".
- Without staging, my mental model had to be the testing strategy. I read every adjacent file before touching `app.js` — route ordering, middleware stacking, DB pool selection, NY anchor helper.

- I documented the change inside the codebase (the schema.sql dated block) so the US-side operator could apply the index without a synchronous call.

## 14. Architectural Decisions

The endpoint follows the existing **route → controller → service** triplet established by other modules (e.g. `rts.routes.js → rts.controller.js → rts.service.js`). Each layer has one responsibility:

- **Route**: declares HTTP verb, mounts the role guard, hands off to the controller.
- **Controller**: shapes HTTP ↔ service; centralises error mapping; logs.
- **Service**: contains all validation, SQL, and business logic; testable without an Express request.

### 14.1 Architecture Flow


## 15. API Design Decisions

- **Two endpoints, not one**: a separate `/count` path so callers that only need the cheap aggregate don't pay the page cost.
- **NY-default date**: optional `date` param; if omitted, the server computes today's NY business date.
- **Camel and snake accepted** for `invoice_type` / `invoiceType` and `includeFormJson` / `include_form_json` — small ergonomic win for curl-from-terminal callers.
- **Both `success` and `ok`** in the response envelope to honour the user-specified contract *and* the rest of the codebase's existing shape.

## 16. Database Query Optimization

| Aspect | Before (codebase default) | After (this work) |
|---|---|---|
| Predicate | `DATE(created_at) = ?` | `created_at >= ? AND created_at < ?` |
| Index usage | Not sargable; full scan | btree range scan on `idx_ia_created_at` |
| Plan stability | Sensitive to row count | Constant work per page |
| Concurrent writes | Contends with INSERTs at scale | Non-blocking |

## 17. Authentication Strategy

- Reused `requireRole(['admin','manager'])` from `middleware/requireRole.js`.
- Role is re-read from `cfc_users` on every request; `x-user-role` header is ignored.
- Identity is accepted in three shapes (header / query / body) — same convention as every other CFC route.
- No JWT, no cookies, no session store added — explicit non-goal.

## 18. Error Handling Strategy

- Validation errors are tagged with `.status` + `.code` and surfaced to the caller with a human-readable `message`.
- Everything else collapses to a 500 with a generic message — the server log holds the stack.
- One centralised `handleArchiveError` in the controller so the two handlers stay symmetric.

## 19. Production Risk Mitigation

- Additive routes only; no existing endpoint touched.
- Idempotent index migration with an `information_schema` existence check.
- Dual `success` / `ok` response shape — no consumer is forced to change.
- Hard caps on `limit` (200) and explicit `includeFormJson=false` escape hatch.
- Documented in `schema.sql` per the team's dated-block convention so the operator deploying the index doesn't need a synchronous walk-through.

## 20. Monitoring Strategy

- Structured log lines mirror the existing module prefix ( `[InvoicesArchive] ...` ) so log search tooling continues to surface them under the same filters.
- Counts in the log enable on-the-fly spotting of unusual volumes (e.g., a day with 0 results when the dashboard shows 50).
- If a future SLO is needed, the request log is sufficient to derive p50/p95 without code changes.

## 21. Testing Strategy

- **Static**: `node --check` on all four touched files; ESM imports resolve cleanly.
- **Manual smoke**: 401 with no userId, 403 as employee, 200 as admin/manager.
- **Negative**: malformed date, impossible calendar date, SQL-injection-shaped `invoice_type` all reject with structured 400s.
- **Adjacency**: confirmed no regression on `/api/reports/today/individual`, `/api/rts/create`, `/api/invoice/save`.
- **Plan verification**: post-index `EXPLAIN` against the new range query expected to show `Using index condition` on `idx_ia_created_at` rather than `ALL`.

## 22. Rollback Strategy

- Code rollback = revert two lines in `app.js`; the new files become dead code.
- Index rollback = `ALTER TABLE invoices_archive DROP INDEX idx_ia_created_at;` — but the index is beneficial to other queries and probably should stay.
- No data was migrated, so rollback is a pure DDL/code operation with no data implications.

# 23. STAR Method — Per-Decision Walkthrough

## 23.1 Choosing range filter over DATE(column)

**SITUATION**

The rest of the codebase used `DATE(created_at) = ?`. It worked for tiny daily reports but would not scale for a generic day-scan API.

**TASK**

Pick a predicate form that stays fast as the table grows and does not require rewriting existing callers.

**ACTION**

Switched to `created_at >= ? AND created_at < nextDay(?)`. Added a btree index `idx_ia_created_at` via an idempotent migration. Left existing callers untouched.

**RESULT**

Day-scan latency moves from full-table-scan dependent to bounded range scan; existing daily reports continue to work without modification.

## 23.2 Reusing requireRole instead of inventing an API key

**SITUATION**

The internal-only constraint could have been satisfied by a new `x-api-key` mechanism, but CFC already has a working role model in `cfc_users`.

**TASK**

Decide between bolting on a new auth surface vs. composing with the existing one.

**ACTION**

Reused `requireRole(['admin','manager'])` verbatim. No new credentials, no key-rotation policy to write, no audit-log split.

**RESULT**

Single source of truth for who is "internal". When an admin is offboarded in `cfc_users`, their API access disappears the same moment.

### 23.3 Anchoring "today" to America/New_York

**SITUATION**

My dev clock is IST; the production server may be UTC; the data is NY-local. A naïve `new Date()` would tip across the business day for half the calls.

**TASK**

Make "today" deterministic and consistent with the rest of CFC's daily jobs.

**ACTION**

Replicated the same `Intl.DateTimeFormat('en-CA', {timeZone:'America/New_York'})` helper used by the daily summary orchestrator. Kept an explicit `date=` override for callers who do want a non-NY day.

**RESULT**

The API, the cron job, and the email report all agree on "today". A whole class of off-by-one bug ceases to exist.

### 23.4 Pagination + includeFormJson toggle

**SITUATION**

`form_json` is LONGTEXT and rows are routinely 50 KB+.

**TASK**

Bound the worst-case response size deterministically.

**ACTION**

Limit clamped to [1, 200] with default 50; `includeFormJson=false` as the "scan a whole day" escape hatch. Two-query fan-out via `Promise.all` instead of the deprecated `SQL_CALC_FOUND_ROWS`.

**RESULT**

Predictable bandwidth/memory profile and no surprise OOM on a busy day.

### 23.5 Centralised error mapping in the controller

**SITUATION**

Two handlers, multiple validation surfaces, easy to drift apart.

**TASK**

Keep validation responses uniform across both endpoints.

**ACTION**

A single `handleArchiveError` helper inspects `err.status` + `err.code`, surfaces 4xx with the original message, and downgrades anything 5xx to a generic string.

**RESULT**

Two handlers, one error contract; adding a third endpoint later is one line of glue.

## 24. How I Thought Like a Production Engineer

- I read the codebase before I wrote anything — conventions are a contract, not a suggestion.
- I asked "what breaks if this is called at the worst possible time?" for every code path: a full-day scan with `form_json`, a malformed date at midnight ET, a deactivated admin who still has a stale token.
- I prefer additive change. The whole feature ships in new files plus a two-line wiring edit.
- I designed the rollback before I designed the rollout.

## 25. How I Minimized Production Risk

- No new dependencies, no new env vars, no new auth model.
- Index DDL packaged behind an existence check so an accidental re-run is a no-op.
- Dual response shape so I never force a consumer change.
- Hard caps on every numeric input — limit clamp, offset floor, regex on invoice_type.
- Stack traces are server-side only.

## 26. How I Designed for Maintainability

- Matched the existing folder triplet ( `routes/` → `controllers/` → `services/` ) so the next engineer needs zero ramp-up.
- Wrote dense module-top comments explaining *why*, not *what*.
- Exported a `__internals__` object so future unit tests can hit the helpers without firing up Express.
- Wrote a dated `schema.sql` block following the team's documented convention, so the next time the DDL is reviewed, this change is in the same shape as every other.

## 27. What a Staff Engineer Would Care About

- **Blast radius**: Is this change reversible in under a minute? Yes.
- **Operational footprint**: Does this add a new on-call surface? No — it reuses the existing logging shape.
- **Cross-team contract**: Does this change a response shape any other service depends on? No — additive keys only.
- **Cost of being wrong**: What if the index is misnamed? The migration is idempotent and the second deploy is a no-op.

## 28. Common Interview Questions & Strong Answers

**Q. Why is `DATE(created_at) = ?` bad?**

It wraps the indexed column in a function, which means MySQL has to evaluate `DATE()` on every row before it can decide whether the row matches. A plain btree on `created_at` can't be used as a starting point for the scan — the predicate is not sargable. The range form `created_at >= ? AND created_at < ?` compares the raw indexed value, so MySQL can seek to the lower bound and stop at the upper bound. On a multi-million-row table that's the difference between a full scan and a thin range scan.

**Q. Why parameterised queries?**

Two reasons: SQL injection safety, and plan-cache reuse. Parameter binding separates the structure of the query from its values — the driver sends them as distinct fields to MySQL, so any value the caller supplies is treated as data, never as SQL. The plan cache benefits because the same query text re-binds many parameter sets; MySQL can keep a single plan.

**Q. Why are internal APIs safer than sharing the DB?**

An API exposes only the operations the engineering team has explicitly designed and reviewed. A DB credential gives the holder the full schema — including write paths. APIs add a logging boundary, a validation boundary, and a contract boundary. They also evolve independently of the schema, so callers don't break when columns are added or renamed.

**Q. Why does indexing matter here?**

Without an index on `created_at`, the day-scan would do a sequential read of the entire `invoices_archive` table. That's linear in retention; it doesn't matter how few rows the day actually has. With a btree on `created_at`, the optimiser seeks straight to the day's segment of the index and reads only those rows.

**Q. Why centralised error handling?**

If every handler maps errors itself, drift is inevitable — one endpoint returns `{ok:false}` on a 400, another returns `{error:"..."}`. A single helper means consumers can write one error-handling branch. It also gives me one place to scrub stack traces before they ever reach the wire.

**Q. Why does observability matter for a small endpoint like this?**

Because diagnostic latency dominates total incident time. A single structured log line with caller, date, count, and outcome lets me answer "did anyone hit this endpoint when reports went weird?" without spinning up an investigation. Cheap to write, expensive to retrofit.

**Q. Why backward compatibility?**

In a multi-consumer system, response shape is a contract. Removing a key is a breaking change even if no current consumer reads it — someone is one git pull away from depending on it. Adding keys (like `success` alongside the existing `ok`) is the cheap way to migrate forward without breaking anyone.

**Q. Why separate authentication from authorization?**

Authentication answers "who are you?"; authorization answers "are you allowed to do this?". Coupling them — for example, treating "has a valid token" as "can do anything" — collapses the two and makes role changes invisible. Re-reading the role from `cfc_users` on every request means an offboarded admin loses access the moment their `is_active` flips, no matter what token they hold.

## 29. Deep-Dive Discussion Points

- **Sargability** — can lead a 20-minute conversation on query optimisation, function-on-column, and predicate transformation.
- **Time-zone correctness** — "wall clock vs instant" is a classic distributed-systems trap.
- **Idempotent migrations** — how to write DDL that can be applied twice without operator stress.
- **Pagination semantics** — limit/offset vs keyset; why I started with the simpler form and what would push me to keyset.
- **Defensive vs paranoid validation** — where to stop "what if the column lies?" parsing.
- **Audit logging as a first-class feature**, not an after-thought.

## 30. Things That Make This Project FAANG-Level

- **Ownership mindset** — I owned not just the code but the migration plan, the rollback plan, and the documentation.
- **Production safety** — additive change, idempotent DDL, dual response shape, hard input caps.
- **Scalability thinking** — index choice and pagination cap are sized for the table's growth, not just today.
- **Operational excellence** — structured logs, no stack-trace leaks, log shape matches the existing module prefix convention.
- **Reliability mindset** — legacy bad data falls back instead of throwing; empty days return `200` with `data:[]`.
- **Debugging strategy** — every log line carries enough context (caller, date, size) to start an investigation cold.
- **Deployment awareness** — designed for one-commit ship across a 12.5-hour gap, with rollback steps documented before rollout.

## 31. Lessons Learned

- The biggest engineering wins come from reading first, writing second. The whole feature ships in a few hundred lines because I composed with existing conventions instead of inventing parallel ones.
- "Default to the existing pattern, deviate only with a written reason" is a faster heuristic than designing fresh.
- An idempotent migration is worth a small upfront cost; an operator never has to remember "did we run this already?"

## 32. Future Improvements

- Cursor (keyset) pagination based on `(created_at, id)` for very large pages.
- ETag / 304 caching for historic days that are never going to change.
- OpenAPI / Swagger publication of the contract.
- Composite index `(invoice_type, created_at)` if the type-filtered path becomes hot.
- Write-side audit log (who queried which day) for compliance.

---

Prepared as part of an SDE1 interview portfolio. All architecture statements reflect the real implementation in the CFC backend repository; quantitative latency numbers, where shown, are explicitly framed as expected outcomes rather than measured benchmarks.